

Experiment 29

Write a C program to simulate the solution of Classical Process Synchronization Problem.

C Program

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t mutex;

void *process(void *arg) {
    int id = *(int *)arg;

    printf("Process %d is waiting for the critical section\n", id);
    sem_wait(&mutex);

    printf("Process %d entered the critical section\n", id);
    sleep(1);
    printf("Process %d is executing the critical section\n", id);
    printf("Process %d leaving the critical section\n", id);

    sem_post(&mutex);
    pthread_exit(NULL);
}

int main() {
    pthread_t t[3];
    int id[3] = {1, 2, 3};

    sem_init(&mutex, 0, 1);

    for (int i = 0; i < 3; i++)
        pthread_create(&t[i], NULL, process, &id[i]);

    for (int i = 0; i < 3; i++)
        pthread_join(t[i], NULL);

    sem_destroy(&mutex);
    printf("Classical process synchronization completed successfully.\n");
    return 0;
}
```

Sample Output

```
Process 1 is waiting for the critical section
Process 1 entered the critical section
Process 2 is waiting for the critical section
Process 3 is waiting for the critical section
Process 1 is executing the critical section
Process 1 leaving the critical section
Process 2 entered the critical section
Process 2 is executing the critical section
Process 2 leaving the critical section
Process 3 entered the critical section
Process 3 is executing the critical section
Process 3 leaving the critical section
Classical process synchronization completed successfully.
```

Result

The classical process synchronization concept was successfully simulated using a semaphore to protect the critical section.